

Comparator Theory: A Working Reference

The concepts behind writing custom comparators are the same across languages; only the syntax for plugging a comparator into a sort call changes. This doc covers the theory you need before working through the practice questions: the comparator contract, ordering rules a comparator must satisfy, stability, and the common patterns and pitfalls.

1. What a Comparator Is

A comparator is a function that takes two elements, *a* and *b*, and returns a signal describing their relative order. Sorting algorithms call it repeatedly instead of using a language's built-in less-than operator, which lets you define “less than” for arbitrary types and arbitrary criteria.

The near-universal convention (three-way comparison) is:

- Return a negative number → *a* should come before *b*.
- Return zero → *a* and *b* are considered equivalent for ordering purposes.
- Return a positive number → *a* should come after *b*.

JavaScript's `Array.prototype.sort(compareFn)`, Java's `Comparator<T>.compare(a, b)`, Python's `functools.cmp_to_key(cmp)`, and C's `qsort` comparator all follow this exact convention. C++'s `std::sort` takes a different shape — a strict-weak-ordering predicate that returns a boolean for “*a* is strictly before *b*” — covered in section 4.

2. The Ordering Contract (Why Your Comparator Can Silently Break Sorting)

A comparator isn't just “any function that returns negative/zero/positive.” To produce a correct, deterministic sort, it must define a strict weak ordering. Concretely, treating `cmp(a,b) < 0` as “*a* < *b*”, the comparator must satisfy:

- Irreflexivity: *a* is never less than itself — `cmp(a, a) == 0`.
- Asymmetry / consistency: if `cmp(a,b) < 0` then `cmp(b,a) > 0` (and vice versa).
- Transitivity: if *a* < *b* and *b* < *c*, then *a* < *c*.
- Transitivity of equivalence: if *a* and *b* compare equal, and *b* and *c* compare equal, then *a* and *c* must also compare equal (this is the one people violate most often when tie-breaking is inconsistent).

If any of these break, the sort's behavior is undefined: languages may throw (Java's `TimSort` detects some contract violations and throws `IllegalArgumentException`), silently produce a

wrong order, or even loop/crash in native implementations. A classic bug: comparing floating-point values with a tolerance (“close enough counts as equal”) breaks transitivity of equivalence, because $a \approx b$ and $b \approx c$ does not imply $a \approx c$.

3. Stability

A sort is stable if elements that compare equal keep their original relative order. Stability matters whenever you sort the same data more than once by different keys (e.g., sort by name, then stable-sort by department — within each department, names stay alphabetical) or when “equal” elements carry meaning beyond the sort key (e.g., preserving insertion order for equal priorities).

- JavaScript: `Array.prototype.sort` is required to be stable as of ES2019 (all modern engines).
- Python: `sorted()` and `list.sort()` are guaranteed stable (Timsort).
- Java: `Collections.sort` / `Arrays.sort` on `Object[]` use a stable, modified mergesort (TimSort). `Arrays.sort` on primitive arrays uses dual-pivot quicksort and is NOT stable — but primitives have no identity beyond value, so it doesn't matter there.
- C++: `std::sort` is NOT guaranteed stable. Use `std::stable_sort` when you need that guarantee.

4. C++'s Comparator Shape Is Different

`std::sort`, `std::stable_sort`, and comparator-based containers (`std::set`, `std::priority_queue`, `std::map`) expect a predicate `comp(a, b)` that returns true if a should come strictly before b, and false otherwise — not a three-way negative/zero/positive value. The predicate must still define a strict weak ordering (never `comp(a,a) == true`; if `comp(a,b)` then not `comp(b,a)`; transitive).

```
// C++: sort vector of pairs by second asc, first desc as tiebreak
std::sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
    if (a.second != b.second) return a.second < b.second;
    return a.first > b.first;
});
```

Note `std::priority_queue` is a MAX-heap by default with the same `std::less`-style comparator — the comparator answers “should a be popped after b”, which is the reverse of how you might first expect it.

5. Comparator vs Comparable / key= vs cmp_to_key

Two ways to tell a sort how to order things: give it a `Comparable` type (the type itself knows how to compare instances of itself), or supply an external `Comparator`/comparator function/key function at call time. External comparators are more flexible — you can sort the same objects different ways in different places without modifying the class.

- Java: implement `Comparable<T>` and override `compareTo(other)` for a type's “natural order”; pass a separate `Comparator<T>` to `sort()` for anything else, including one-off or multi-field orders.
- Python: prefer `key=` with a function/lambda that returns a sortable tuple, e.g. `key=lambda p: (-p.age, p.name)`. This is faster than a comparator because the key is computed once per element ($O(n)$), not once per comparison ($O(n \log n)$). Only fall back to `functools.cmp_to_key(cmp)` when the ordering genuinely can't be expressed as a single sortable key (e.g., comparing string concatenations `a+b` vs `b+a` for the ‘Largest Number’ problem).
- JavaScript: `sort(compareFn)` is the only built-in mechanism; there's no separate `key=` shortcut, though you can simulate the same optimization manually (see the next section).

6. Decorate-Sort-Undecorate (a.k.a. Schwartzian Transform)

When the sort key is expensive to compute (e.g., parsed from a string, or the result of a formula), computing it inside the comparator recomputes it $O(n \log n)$ times. Instead, compute it once per element, sort by the precomputed value, then discard it:

```
// JS: expensive key computed once instead of on every comparison
const decorated = items.map(item => [expensiveKey(item), item]);
decorated.sort((a, b) => a[0] - b[0]);
const sorted = decorated.map(pair => pair[1]);
```

This is exactly what Python's `key=` does for you automatically under the hood.

7. Multi-Field (Composite) Comparators

The standard pattern for “sort by A, then by B, then by C” is to compare by the first field and, only if it's a tie (zero), fall through to the next field. Chain the comparisons left to right in priority order, using `||` in JS or early-return in other languages:

```
// JS: department asc, then salary desc, then name asc
employees.sort((a, b) =>
  a.department.localeCompare(b.department) ||
  b.salary - a.salary ||
  a.name.localeCompare(b.name)
);

// Java: same idea via Comparator chaining
employees.sort(
  Comparator.comparing((Employee e) -> e.department)
    .thenComparing(Comparator.comparingDouble((Employee e) ->
      e.salary).reversed())
    .thenComparing(e -> e.name)
);
```

For an ascending/descending mix in Python, encode direction directly in the key tuple (negate numeric fields to flip them, since strings can't be negated):

```
employees.sort(key=lambda e: (e.department, -e.salary, e.name))
```

8. Common Pitfalls

- Subtraction shortcuts overflow: $a - b$ as a numeric comparator is a common trick, but for very large or very negative integers (near `INT_MIN`/`INT_MAX` in fixed-width languages like Java/C++) the subtraction can overflow and silently flip the sign. Prefer `Integer.compare(a, b)` / explicit if/else comparisons in those languages.
- Non-transitive tie-breaks: chaining conditions that don't reduce to a total order (e.g., paper-scissors-rock-style rules) will corrupt the sort unpredictably depending on pivot choices.
- Inconsistent null/undefined handling: decide once where nulls sort (usually last) and make every comparison agree — comparing null against every other value the same way.
- Locale-sensitive string comparison: using `<` / `>` on strings is code-point order, not human alphabetical order, and is case-sensitive (all uppercase letters sort before all lowercase). Use `localeCompare` (JS), `Collator` (Java), or `locale.strcoll` / `casefold` (Python) when “alphabetical” must match human expectations.
- Mutating comparator state mid-sort (e.g., a counter used inconsistently) can violate the ordering contract from one call to the next.
- Forgetting stability requirements: if downstream logic depends on ties keeping original order, confirm the language's sort is actually stable (see section 3) rather than assuming it.

9. Comparators in Heaps / Priority Queues

Binary heaps use the same three-way (or boolean, in C++) comparator to decide parent/child ordering instead of a full sort. Two consequences show up constantly in interview problems:

- Min-heap vs max-heap is just a sign flip on the comparator: $\text{cmp}(a,b) = a - b$ gives a min-heap; $\text{cmp}(a,b) = b - a$ gives a max-heap. Python's `heapq` is min-heap only, so max-heap behavior is typically faked by negating numeric values on push/pop.
- Heaps are not stable by default. If insertion order should break ties, encode a monotonically increasing counter into the comparator as the final tiebreaker (push tuples like `(priority, insertion_index, item)`).

10. Quick Checklist Before You Submit a Comparator

- Does it return the right sign convention for the language/API you're using (three-way vs boolean)?
- Is it transitive and consistent for every pair, including ties across more than two elements?
- Are all tie-break fields ordered by priority, with the most important comparison first?
- Does it avoid raw subtraction on values that could overflow?

- Does it handle null/missing/undefined values the same way every time?
- If ties matter for order preservation, is the underlying sort/heap actually stable — or do you need to add an explicit tiebreaker?